Documentação — Mapa de Locais (Flutter + Firestore)

Oct 8, 2026 · @Lucas Terminiello

1. Descrição da problemática

Uma empresa quer um aplicativo para que as pessoas encontrem serviços e estabelecimentos em diferentes cidades. O usuário precisa escolher a cidade, pesquisar o que procura, filtrar os resultados, abrir os detalhes de um local e marcar favoritos.

Além de consultar, o sistema precisa manter os dados: cadastrar novos locais, alterar e excluir os existentes. Por isso os locais não podem ficar fixos no código; eles ficam em um banco de dados na nuvem, compartilhado por todos os aparelhos.

2. Objetivo da aplicação

Desenvolver um aplicativo Android em Flutter que permita localizar estabelecimentos em São Paulo, Campinas e Jundiaí, com os dados armazenados no Cloud Firestore.

O aplicativo tem três telas principais (Mapa, Resultados e Detalhes) e permite:

- selecionar a cidade e ver os locais dela como marcadores em um mapa;
- pesquisar por nome ou categoria e filtrar por cidade, categoria e favoritos;
- cadastrar, editar e excluir locais, com confirmação antes de excluir;
- consultar todas as informações de um local e marcá-lo como favorito.

3. Requisitos funcionais

Os dez requisitos do enunciado foram implementados; a coluna "Onde" indica a tela e o arquivo principal de cada um.

Código	Requisito	Como foi atendido	Onde
RF01	Selecionar cidade	Seletor com São Paulo, Campinas e Jundiaí (e "Todas" nos resultados), em Bottom Sheet	Tela 1 e Tela 2 · `widgets/seletor_cidade.dart`

Código	Requisito	Como foi atendido	Onde
RF02	Pesquisar	Busca por nome ou categoria, sem diferenciar maiúsculas e acentos	Tela 1 e Tela 2 · `utils/filtros.dart`
RF03	Consultar Firestore	Lista em tempo real com `snapshots()` da coleção `locais`	`services/local_service.dart`
RF04	Filtrar	Chips de categoria e favoritos + seletor de cidade, combináveis	Tela 2 · `utils/filtros.dart`
RF05	Cadastrar local	Formulário validado em Bottom Sheet; grava com `add()`	Tela 2 · `widgets/local_form.dart`
RF06	Editar local	Mesmo formulário, já preenchido; grava com `update()` pelo ID	Telas 2 e 3
RF07	Excluir local	Diálogo de confirmação; só exclui ao tocar em "Excluir"	Telas 2 e 3 · `dialogs/confirm_delete.dart`
RF08	Detalhes	Imagem, nome, categoria, avaliação, distância, horário (com aberto/fechado), descrição, cidade e favorito	Tela 3 · `screens/detalhes_screen.dart`
RF09	Favoritar	Coração altera o campo `favorito` do documento	Tela 3
RF10	Navegação	Mapa → Resultados → Detalhes, voltar, "Ver no mapa" e barra inferior	Todas as telas

4. Requisitos não funcionais

Requisito	Como foi atendido
Desenvolvido em Flutter/Dart	Projeto Flutter, plataforma Android

Requisito	Como foi atendido
Banco Cloud Firestore	Coleção `locais`, acessada pelo pacote `cloud_firestore`
Configurado com FlutterFire	`flutterfire configure` gera `lib/firebase_options.dart`
Funcionar em Android	Projeto criado e testado para Android (minSdk 24)
Código organizado em pastas	`models`, `services`, `screens`, `widgets`, `dialogs`, `utils`
Sem erros na demonstração	`flutter analyze` sem avisos; estados de carregando, vazio e erro tratados em todas as telas
Exclusão pede confirmação	Diálogo "Excluir local?" com Cancelar e Excluir
Mensagens de sucesso ou erro	SnackBar verde no sucesso e vermelha no erro; erros do Firebase traduzidos (ex.: sem permissão, sem internet)

5. Tecnologias utilizadas

Tecnologia	Versão	Uso
Flutter / Dart	3.47.4 / 3.13.3	Interface e lógica do aplicativo
firebase_core	^4.15.0	Inicialização do Firebase
cloud_firestore	^6.10.0	Leitura e gravação da coleção `locais`
firebase_auth	^6.7.0	Login anônimo exigido pelas regras de segurança
Firebase CLI + FlutterFire CLI	firebase-tools 15.33.0 · flutterfire_cli 1.4.1	Configuração do projeto Firebase no Flutter
Cloud Firestore	—	Banco de dados NoSQL (coleções e documentos)
flutter_test	SDK	Testes automatizados de modelo, filtros e interface
Emulador do Firestore	—	Testes das regras de segurança

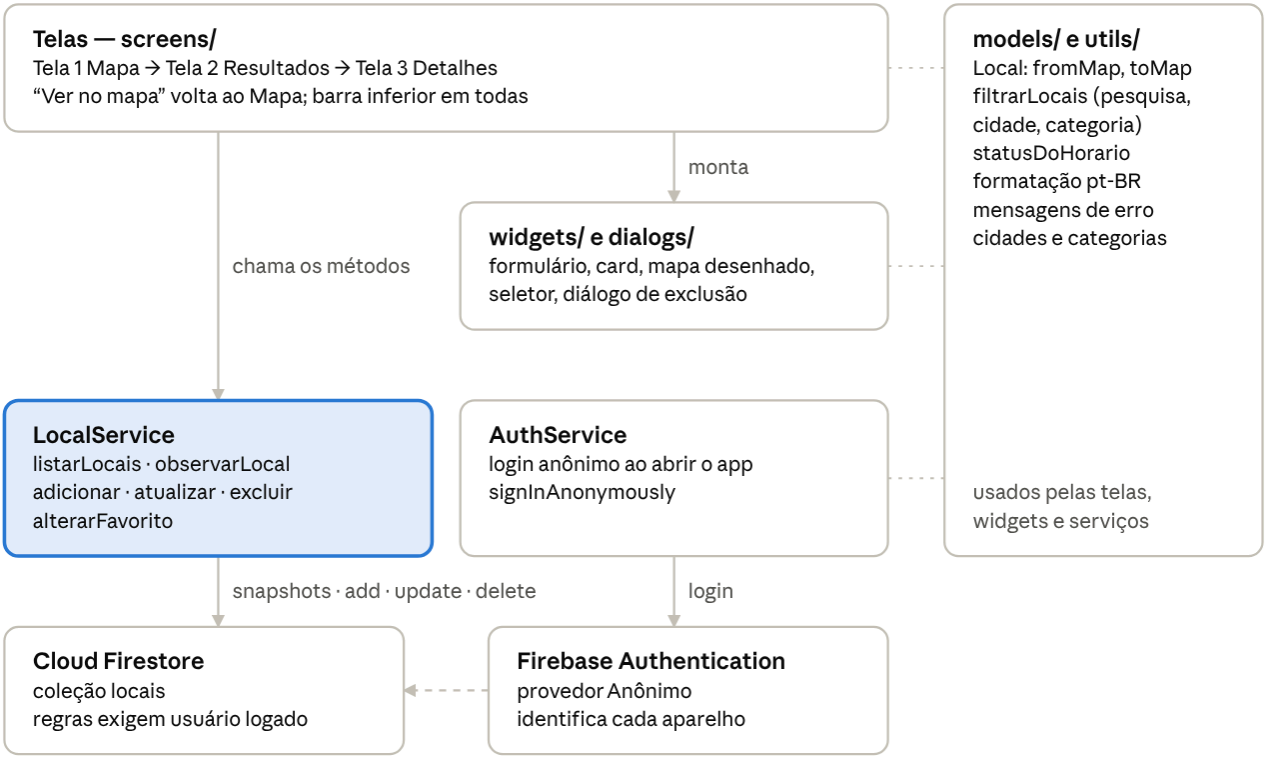
O mapa da Tela 1 é uma representação desenhada com `CustomPainter`; não foi usado Google Maps (extensão opcional).

6. Arquitetura do projeto

O código é dividido em camadas: as telas montam a interface e pedem dados aos serviços, e só o `LocalService` conversa com o Firestore.

As telas só acessam o Firestore pelo LocalService

Camadas do projeto, da interface (topo) ao Firebase (base)



arquitetura em camadas · telas, interface, serviços e Firebase

O formulário e o diálogo de exclusão não acessam o banco: recebem da tela a função que salva ou devolvem a confirmação. Por isso podem ser testados sem Firebase.

```
lib/
├── main.dart              inicializa Firebase e login anônimo
├── firebase_options.dart  gerado pelo FlutterFire
├── models/                local.dart
├── services/              local_service, auth_service, dados_exemplo
├── screens/               mapa, resultados, detalhes
├── widgets/               local_card, local_form, mapa_representacao,
│                          seletor_cidade, barra_inferior, chip_filtro,
imagem_local
├── dialogs/               confirm_delete.dart
└── utils/                 filtros, horario, formatadores, mensagens,
constantes
```

7. Modelo da coleção locais

Cada local é um documento da coleção `locais` com 8 campos. O ID do documento é gerado pelo Firestore e não é gravado dentro dele; o modelo `Local` guarda esse ID para editar, excluir e favoritar.

Campo	Tipo	Exemplo	Validação no formulário
nome	String	"Academia Fitness"	Obrigatório, até 60 caracteres
categoria	String	"Academia"	Academia, Restaurante, Hospital ou Farmácia
cidade	String	"São Paulo"	São Paulo, Campinas ou Jundiaí
avaliacao	Number	4.8	Entre 0 e 5
distancia	Number	1.2	Em km, de 0 a 1000
horario	String	"06:00 - 22:00"	Formato HH:MM - HH:MM ou "24 horas"
descricao	String	"Academia completa…"	Até 500 caracteres
favorito	Boolean	false	Começa falso no cadastro

Documento de exemplo:

```
{
  "nome": "Academia Fitness",
  "categoria": "Academia",
  "cidade": "São Paulo",
  "avaliacao": 4.8,
  "distancia": 1.2,
  "horario": "06:00 - 22:00",
  "descricao": "Academia completa com musculação, funcional, aulas coletivas
e personal trainer.",
  "favorito": false
}
```

A coleção foi populada com 18 locais (6 por cidade) pelo botão "Carregar locais de exemplo", que aparece quando a coleção está vazia.

8. Operações CRUD

Toda conversa com o Firestore fica na classe `LocalService`; as telas só chamam seus métodos. Assim, os comandos de banco não se espalham pela interface.

Operação	Método do LocalService	Chamada no Firestore	Onde é usada	Mensagem ao usuário
Create	`adicionar(local)`	`locais.add(local.toMap())`	Botão "Novo local" (Tela 2)	"Local cadastrado com sucesso!"
Read (lista)	`listarLocais()`	`locais.snapshots()`	Telas 1 e 2	Carregando, lista vazia ou erro
Read (um local)	`observarLocal(id)`	`locais.doc(id).snapshots()`	Tela 3	"Este local foi excluído."
Update	`atualizar(local)`	`locais.doc(id).update(local.toMap())`	Editar (Telas 2 e 3)	"Local atualizado com sucesso!"
Update parcial	`alterarFavorito(id, valor)`	`locais.doc(id).update({'favorito': valor})`	Coração (Tela 3)	"Adicionado aos favoritos"
Delete	`excluir(id)`	`locais.doc(id).delete()`	Excluir, após confirmação (Telas 2 e 3)	"<nome> foi excluído."

Como as telas escutam `snapshots()`, qualquer operação aparece na hora, sem recarregar. Se o Firestore recusar a operação, o formulário continua aberto com a mensagem de erro, e o usuário não perde o que digitou.

9. Segurança do banco de dados

O banco não fica aberto: só usuários autenticados acessam a coleção `locais`, e toda gravação passa por validação de campos. As regras ficam em `firestore.rules` e foram publicadas no console do Firebase.

- **Autenticação**: ao abrir, o app faz login anônimo (`signInAnonymously`). O usuário não digita nada, mas cada aparelho recebe uma identidade, e as regras exigem `request.auth != null`.
- **Validação**: criar ou editar só é aceito com os 8 campos do modelo, nos tipos certos e dentro dos limites (ex.: avaliação entre 0 e 5, nome com até 60 caracteres). Campos extras ou faltando são recusados.
- **Outras coleções**: qualquer caminho fora de `locais` é bloqueado.

As regras foram testadas no emulador oficial do Firestore (`testes_regras/`): 14 de 14 cenários passaram, entre eles leitura sem login negada, avaliação 7 recusada, campo extra recusado e favoritar permitido.

Limitação: o login anônimo não separa administrador de usuário comum, então qualquer pessoa com o app pode cadastrar e excluir. Separar perfis exigiria login real e permissões por usuário (desafio extra nível 3).

10. Tabela de testes

Os 11 testes obrigatórios são manuais, feitos no aparelho Android; a coluna Resultado é marcada após a execução.

ID	Teste	Procedimento	Resultado esperado	Resultado
T01	Conexão	Abrir o app e depois a tela de Resultados	Locais do Firestore aparecem na lista e como marcadores no mapa	Passou
T02	Pesquisa	Pesquisar "Academia"	Só academias na lista	Passou
T03	Cidade	Selecionar Campinas no mapa	Cabeçalho mostra Campinas e os marcadores da cidade	Passou
T04	Filtro	Tocar no chip "Restaurantes"	Só restaurantes; contador atualizado	Passou
T05	Create	Novo local, preencher e Cadastrar; conferir no console	Mensagem de sucesso e documento novo no Firestore	Passou

ID	Teste	Procedimento	Resultado esperado	Resultado
T06	Read	Fechar e reabrir a tela (ou o app)	O local cadastrado continua na lista	Passou
T07	Update	Editar nome ou horário e salvar	Mensagem de sucesso; lista, detalhes e console com o novo valor	Passou
T08	Delete	Excluir um local e confirmar	Diálogo aparece; o local some e aparece a mensagem; Cancelar mantém o local	Passou
T09	Favorito	Tocar no coração nos detalhes, duas vezes	Coração preenche e esvazia; campo `favorito` muda no console	Passou
T10	Navegação	Mapa, Resultados, Detalhes, voltar; depois "Ver no mapa"	Cada voltar retorna à tela anterior; "Ver no mapa" destaca o marcador	Passou
T11	Erro	(a) Pesquisar "xyz"; (b) publicar `allow read: if false` por um momento	(a) "Nenhum local encontrado"; (b) "Erro ao carregar os locais" com aviso de permissão (restaurar as regras depois)	Passou

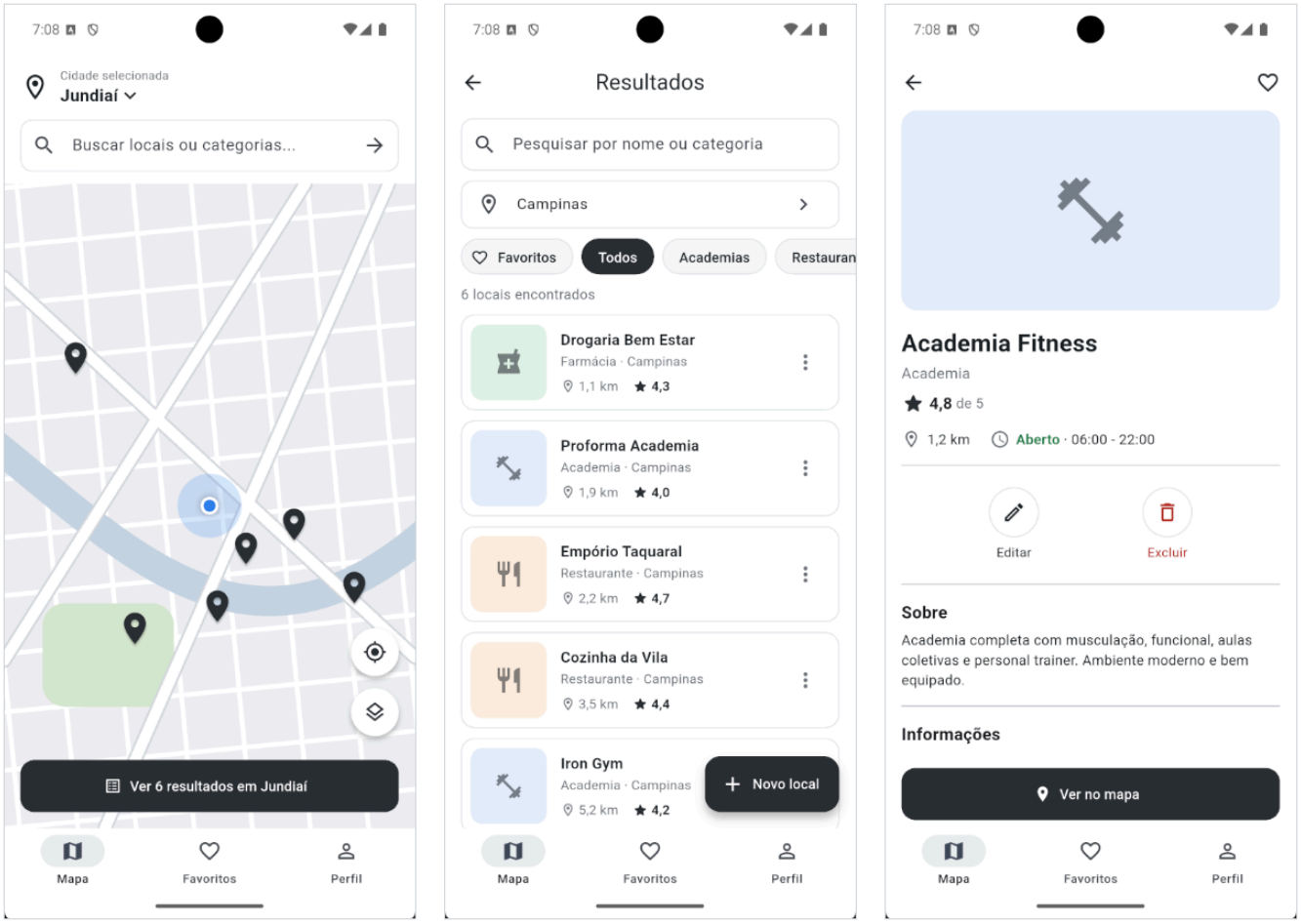
Testes automatizados, executados com `flutter test` (31 de 31 passaram) e com o emulador do Firestore:

Arquivo	O que cobre	Resultado
`test/filtros_test.dart`	Pesquisa, cidade, categoria, favoritos e combinação (T02 a T04 automatizados); cobertura dos dados de exemplo	10 de 10
`test/widget_test.dart`	Card, menu editar/excluir, formulário (validação, cadastro, edição), diálogo de exclusão, posições do mapa	9 de 9
`test/horario_test.dart`	Formato do horário e aberto/fechado, inclusive após a meia-noite	7 de 7
`test/local_test.dart`	Conversão do documento (fromMap/toMap), campos ausentes, copyWith	5 de 5

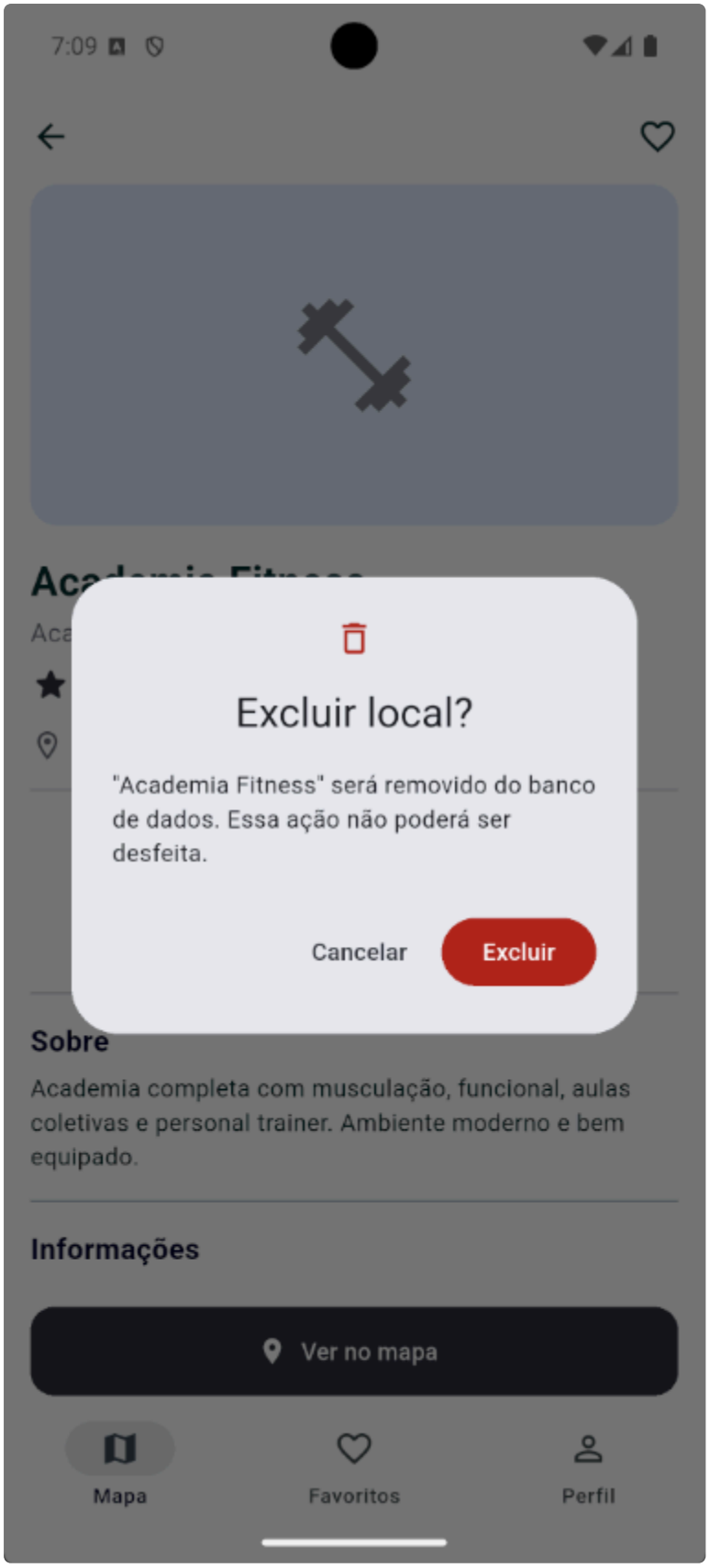
Arquivo	O que cobre	Resultado
`testes_regras/`	14 cenários das regras de segurança	14 de 14

11. Capturas das três telas

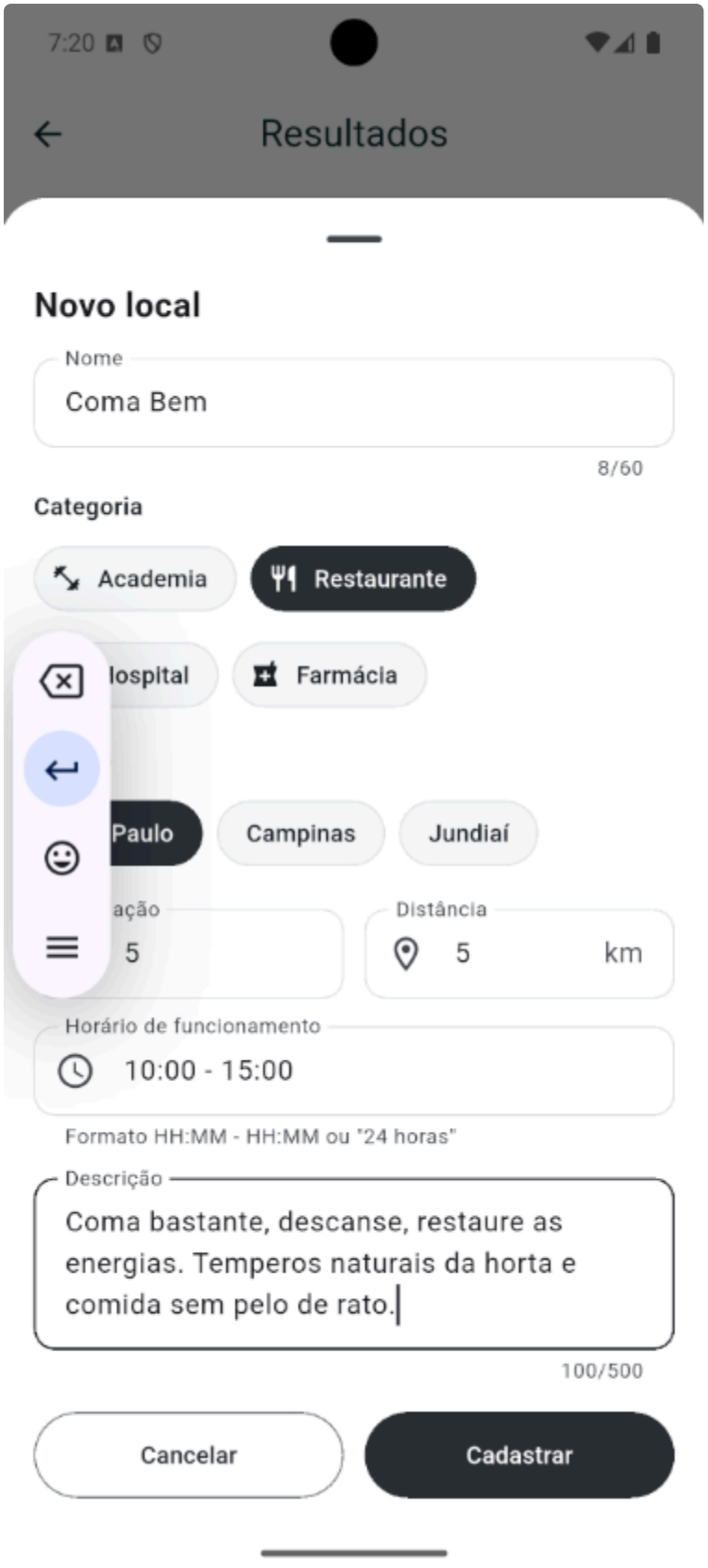
Capturas feitas no emulador Android (Pixel 8, API 35) com o app conectado ao Firestore. Da esquerda para a direita: Tela 1 — Mapa (Jundiaí, 6 marcadores), Tela 2 — Resultados (Campinas, 6 locais) e Tela 3 — Detalhes (Academia Fitness).



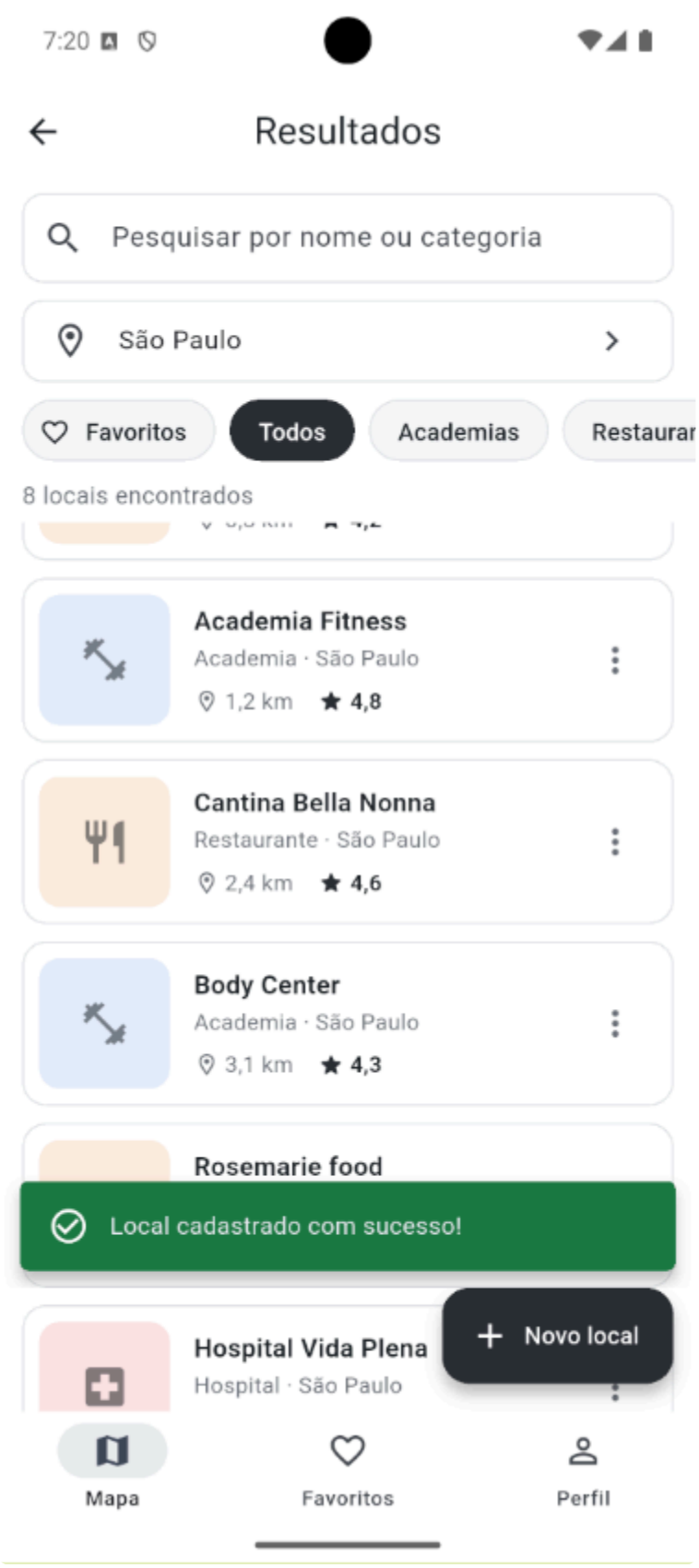
Confirmação de exclusão (diálogo aberto a partir da tela de detalhes)



Formulário de cadastro (Bottom Sheet do botão "Novo local")

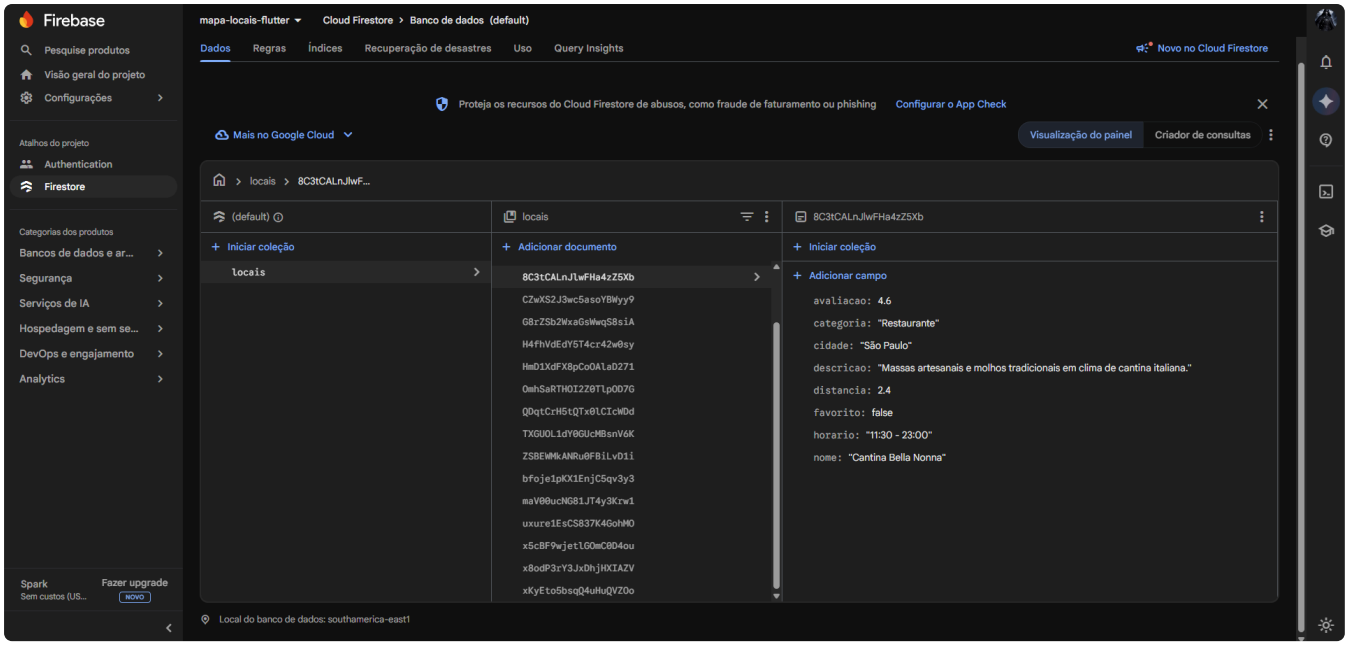


Cadastro concluído (mensagem de sucesso após salvar; evidência do T05)



12. Print do Firestore

Console do Firebase, em Firestore Database, com a coleção `locais` e pelo menos 10 documentos; um deles aberto mostrando os 8 campos.



Banco (default) em southamerica-east1, com o documento "Cantina Bella Nonna" aberto.

13. Conclusão

O aplicativo atende aos 10 requisitos funcionais com dados reais no Cloud Firestore: as três telas leem a coleção em tempo real e o CRUD completo funciona com confirmação e mensagens de retorno.

Três decisões pesaram mais. Concentrar o acesso ao banco no `LocalService` deixou as telas simples e permitiu testar formulário e filtros sem Firebase. Filtrar em memória foi adequado para poucas dezenas de locais, mas com milhares seria preciso usar `where()` e índices no Firestore. Já o modelo sem latitude e longitude obrigou a calcular a posição dos marcadores a partir do ID, o que funciona para uma representação, mas não para um mapa real.

Como evolução, os próximos passos seriam coordenadas no modelo com Google Maps, fotos no Firebase Storage e login real com perfil de administrador, que resolveria a principal limitação de segurança atual.